

Practice Lab #6: Latches + Behavioral Modelling

Lecturer: Harikrishnan N B

Student:

Q1. 2-to-1 Multiplexer

Theory: Draw the block Diagram for a 2-to-1 multiplexer.

Lab: Implement a 2-to-1 mux behaviourally which selects between two 1-bit inputs. (`mux_2to1.v`).

Q2. Full Adder

Lab: Implement behaviorally only. (`full_adder.v`).

Q3. 5-to-32 Decoder

Theory: Draw the block diagram for a 5-to-32 decoder.

Lab: Implement a 5-to-32 decoder behaviorally which activates one of thirty-two outputs based on five input lines. (`decoder_5to32.v`).

Q4. 32-to-5 Encoder

Theory: Draw the block diagram for a 32-to-5 encoder.

Lab: Implement a 32-to-5 encoder behaviorally which generates a 5-bit binary output corresponding to the active input line. (`encoder_32to5.v`).

Q5. D-Latch

Theory: Draw the block diagram for a level-sensitive D latch.

Lab: Implement a D latch behaviorally with inputs D, EN (enable) and output Q. (`d_latch.v`).

Q6. Alien Signal Decoder – Tactical Routing Edition

Objective: Design a modular behavioral Verilog system that processes encrypted alien mission codes and routes them based on signal validity and tactical intelligence. The system must decode, validate, analyze, store, and route signals using a structured hierarchy of modules.

System Overview: The system receives a 5-bit alien mission code and performs the following operations:

- Decodes the mission code into a 32-bit one-hot signal.
- Validates the signal based on its lower 4 bits.
- Extracts intelligence from the middle 8 bits of the signal.
- Stores the lower 8 bits in a level-sensitive latch.
- Routes one bit from the latched signal based on tactical priority.
- Encodes the latched signal into a zone ID.
- Raises an alert if the signal is invalid or lacks intelligence.

Module Implementation: You must implement the following modules in separate Verilog files:

- `decoder_5to32.v` Accepts a 5-bit input and outputs a 32-bit one-hot signal.
- `power_of_two_checker.v` Accepts 4 bits and outputs a high signal if the input represents a valid one-hot pattern. Do not try hard coding values over here since for larger values it may not work. Try to implement using the given modules. (`adder_3bit.v`)
- `bit_intel_extractor.v` Accepts an 8-bit input (middle byte of the decoded signal) and outputs a 3-bit intelligence score. The score must be derived by analyzing three distinct regions of the input:
 - **Top 2 bits (bits 7 and 6):** Check if both bits are active simultaneously. Contributes to LSB of `intel_score`.
 - **Middle 2 bits (bits 5 and 4):** Check for contrast between the bits. Contributes to center bit of `intel_score`.
 - **Bottom 2 bits (bits 3 and 2):** Check if either bit is active. Contributes to MSB of `intel_score`.

Each of these checks contributes one bit to the final 3-bit intelligence score. The output should be non-zero only when meaningful structure is detected in these regions. This score is used in the top-level module to determine whether the signal contains actionable intelligence.

- `d_latch_8bit.v` Accepts 8 bits and a latch enable signal. Outputs the stored value when enabled.
- `mux_8to1.v` Accepts 8 bits and a 3-bit selector. Outputs the selected bit.
- `encoder_8to3.v` Accepts a one-hot 8-bit input and outputs a 3-bit zone ID.
- `alien_signal_core.v` Top-level module that integrates all the above components. Connects inputs and outputs as described below.

Top-Level Module: Implement `alien_signal_core.v` to connect all modules. Inputs:

- `alien_code[4:0]` – Alien mission code
- `latch_enable` – Controls whether the latch stores data
- `priority_zone[2:0]` – Selector for routing logic

Outputs:

- `routed_bit` – Selected bit from the latched signal
- `alert` – High if the signal is invalid or intelligence score is zero
- `zone_id[2:0]` – Encoded zone ID from the latched signal

Constraints:

- All modules must be implemented as specified in the question.
- Do not use `assign` statements or `for` loops.
- Arithmetic operations must be modularized.
- Structural and behavioral modeling may be mixed.

Debugging Tips

- Always declare intermediate wires before use.
- You are free to implement your own modules and use them.
- Check carefully for missing commas/semicolons.
- Ensure all gate inputs/outputs are explicitly connected.
- If output shows X, recheck wire connections.